AWS Internship Project Documentation

Sep 29, 2026 · @karthi

1. Introduction

This project builds a complete, Free Tier–safe AWS environment: a custom VPC, an EC2 web server, S3 storage, IAM roles, CloudWatch monitoring, an EBS snapshot backup, and Budget alerts, all in one end-to-end workflow.

Title: Custom VPC + EC2 Web Application + S3 Storage + IAM Roles + CloudWatch Monitoring + Automated Backup and Cost Control

Complexity: Moderate. **Constraint:** Free Tier only.

Problem statement

Launching an EC2 instance alone is not enough for a practical deployment. A complete environment also needs:

- network isolation (VPC and security groups)
- least-privilege IAM access (users and roles)
- separation of application hosting (EC2) and storage (S3)
- monitoring and alerting for failures
- backup for recovery
- cost visibility so Free Tier limits are not exceeded

Free Tier approach

Service	How it stays free
Amazon EC2	One t2.micro or t3.micro Linux instance
Amazon EBS	Root volume within the Free Tier GB limit
Amazon VPC	Free components only: VPC, subnet, IGW, route table
AWS IAM	Free (users, groups, roles, policies)
Amazon S3	Small demo files and logs only

Service	How it stays free
Amazon CloudWatch	Basic metrics, a few alarms, limited log ingestion
Amazon SNS	Email notifications
AWS Budgets	Billing and usage alerts
EBS Snapshots	One small demo snapshot

NAT Gateway, Application Load Balancer, RDS, and Auto Scaling are deliberately avoided. Eligibility was verified in AWS Billing → Free Tier before creating resources.

2. Objectives and Scope

The project has eight objectives, each mapped to a phase of the implementation.

1. Design and launch a custom VPC with a public subnet, internet gateway, and route table.
2. Deploy a Free Tier EC2 Linux web server hosting a simple application.
3. Create IAM users, groups, and policies, plus an EC2 instance role for secure S3 access with no hardcoded keys.
4. Use S3 for application assets and log archival.
5. Configure CloudWatch metrics, alarms, and log monitoring with SNS email alerts.
6. Create an EBS snapshot backup and document a restore plan.
7. Enforce cost control using tags, Budgets, and a cleanup checklist.
8. Demonstrate log analysis on a sample dataset of web access logs.

In scope

- One VPC, one public subnet, internet gateway, and route table
- One Free Tier EC2 instance running Apache or Nginx
- Security group hardening (SSH restricted to my IP, HTTP allowed carefully)
- IAM user, group, custom policy, and an EC2 instance role
- S3 bucket for static assets and archived logs
- CloudWatch CPU and status-check alarms, SNS email, and limited CloudWatch Logs
- One EBS snapshot and one AWS Budget alert
- Sample access-log dataset with a simple analysis script

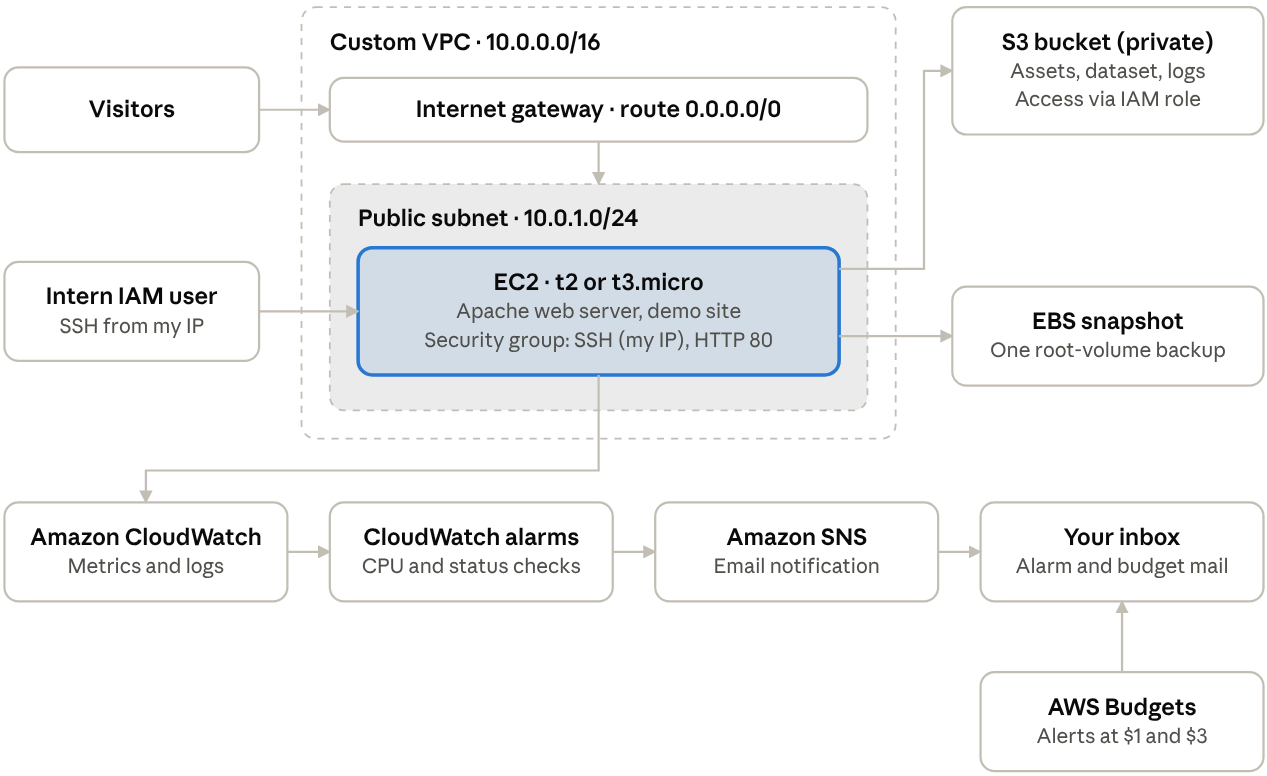
Out of scope

- Multi-AZ high availability
- Application Load Balancer and Auto Scaling
- RDS or DynamoDB production databases
- NAT Gateway and private-subnet application tiers (cost risk)
- Kubernetes, ECS, and EKS

3. Architecture Overview

The design centres on one EC2 web server in a public subnet, with IAM, S3, CloudWatch, SNS, snapshots, and Budgets each attached to it for one purpose.

One EC2 server ties IAM, S3, monitoring and backup together



architecture · EC2 server with IAM, S3, monitoring, backup and budget

Visitors reach the site through the internet gateway, and I reach it over SSH from my own IP. The instance uses its IAM role to read and write S3, sends metrics to CloudWatch, and alarms notify me by SNS email. A snapshot backs up the root volume, and a Budget warns me about spend.

4. Phase 1: IAM Foundation

All work is done through a least-privilege IAM user, never the root account, and the EC2 instance reaches S3 through a role instead of stored access keys.

Steps

1. Enable MFA on the root account, then create a daily-use IAM user.
2. Create groups: `InternDevelopers` and `CloudOperators`.
3. Attach least-privilege policies for EC2, a single S3 bucket, CloudWatch, and SNS.
4. Create an EC2 instance role (trusted entity: EC2) with access to the project bucket only.
5. Attach the role to the instance through an instance profile.

Identity summary

Identity	Type	Purpose
Root account	Account owner	MFA enabled, used only for initial setup
`intern-user`	IAM user	Daily console and CLI work
`InternDevelopers`	IAM group	EC2, S3 (bucket-scoped), CloudWatch permissions
`CloudOperators`	IAM group	Monitoring, SNS, and Budgets permissions
`EC2-S3-Access-Role`	IAM role	Lets the instance read and write the project bucket

The names above are suggestions; replace them with the ones used in the console.

EC2 role policy

The role allows only `s3:GetObject`, `s3:PutObject`, and `s3:ListBucket` on the project bucket.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::my-internship-bucket"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::my-internship-bucket/*"
    }
  ]
}
```

Why instance roles beat access keys

- Credentials are temporary and rotate automatically, so there is nothing long-lived to leak.
- No secrets sit in code, config files, or the AMI, and none can be committed to Git.
- Permissions can be changed or revoked centrally without touching the server.
- Access is scoped to one bucket, which limits the damage if the instance is compromised.

5. Phase 2: Custom VPC Networking

The network is a single VPC with one public subnet routed to an internet gateway, which costs nothing and keeps the design simple.

Resource	Configuration
VPC	`10.0.0.0/16`
Public subnet	`10.0.1.0/24`, auto-assign public IPv4 enabled
Internet gateway	Attached to the VPC
Route table	`10.0.0.0/16` → local, `0.0.0.0/0` → internet gateway, associated with the public subnet

Steps

1. Create the VPC with CIDR `10.0.0.0/16`.
2. Create the public subnet `10.0.1.0/24` in one Availability Zone.
3. Create an internet gateway and attach it to the VPC.
4. Create a route table, add the route `0.0.0.0/0` → internet gateway, and associate it with the subnet.
5. Enable auto-assign public IP on the subnet.
6. Tag every resource with `Project=InternshipAWS`, `Env=Demo`, and `Owner=<yourname>`.

Tags make cleanup and cost tracking easy, because everything created for the project can be found with one filter.

Evidence to capture: screenshots of the VPC, subnet, route table (showing the IGW route), and internet gateway.

6. Phase 3: EC2 and Web Application

A single t2.micro or t3.micro instance in the public subnet hosts a small "Student Portal / Internship Demo Site" served by Apache.

Security group

Type	Port	Source	Reason
SSH	22	My IP only (`x.x.x.x/32`)	Never open SSH to `0.0.0.0/0`
HTTP	80	`0.0.0.0/0`	Public demo access

Instance configuration

Setting	Value
AMI	Ubuntu (Free Tier eligible) or Amazon Linux
Instance type	t2.micro or t3.micro
Network	Custom VPC, public subnet
Key pair	Created and stored securely (`my-key.pem`)

Setting	Value
IAM instance profile	`EC2-S3-Access-Role`
Root volume	Within the Free Tier GB limit

Steps

1. Launch the instance with the settings above and the project tags.
2. Connect over SSH: `ssh -i my-key.pem ubuntu@<EC2-PUBLIC-IP>`.
3. Install Apache: `sudo apt update && sudo apt install -y apache2`.
4. Deploy the demo pages: a home page with the project summary and a `/health` page for uptime checks.
5. Load CSS and images from S3, or mirror them from S3 to the web root.
6. Verify with `curl http://127.0.0.1` on the server and by opening the public IP in a browser.
7. Visit the site several times to generate entries in the Apache access log.

```
# SSH
ssh -i my-key.pem ubuntu@<EC2-PUBLIC-IP>

# Web server check
curl http://127.0.0.1
```

Evidence to capture: EC2 instance details, security group rules, browser view of the site, and the `curl` output.

7. Phase 4: Storage (S3 and EBS Snapshot)

S3 holds static assets and archived logs, while one EBS snapshot of the root volume serves as the backup and recovery evidence.

S3 bucket

1. Create a private bucket (for example `my-internship-bucket`) with Block Public Access on and the project tags.
2. Upload static assets (CSS and images) and the dataset `web-access-logs.csv`.

3. From EC2, sync Apache logs to a dated path using the instance role, with no access keys on the server.

```
aws s3 sync /var/log/apache2/ s3://my-internship-bucket/logs/$(date +%F)/
```

Suggested bucket layout:

Path	Contents
`assets/`	CSS and images for the site
`data/`	`web-access-logs.csv`
`logs/yyyy-mm-dd/`	Rotated Apache access and error logs

EBS snapshot backup

1. In EC2 → Volumes, select the root volume of the instance.
2. Choose Create snapshot, add a description and the project tags.
3. Wait until the status is Completed and capture a screenshot.
4. Keep only this one snapshot, since snapshot storage counts toward Free Tier limits.

Restore plan

1. Open EC2 → Snapshots and select the snapshot.
2. Choose Create volume from snapshot in the same Availability Zone as the instance.
3. Stop the instance, detach the old root volume, and attach the new volume as the root device (`/dev/sda1` on Ubuntu).
4. Start the instance, then confirm the site responds and the logs are intact.
5. Alternatively, create an AMI from the snapshot and launch a replacement instance.
6. Delete any test volumes or extra snapshots afterward to avoid charges.

8. Phase 5: Monitoring, Alerts, and Cost Control

Two CloudWatch alarms notify me by SNS email when the instance is overloaded or unhealthy, and an AWS Budget warns before spend becomes a problem.

SNS topic

1. Create an SNS topic (for example `internship-alerts`) with the project tags.
2. Subscribe my email address with the Email protocol.
3. Open the confirmation email and confirm the subscription, then capture a screenshot.

CloudWatch alarms

Alarm	Metric	Condition	Action
High CPU	`CPUUtilization`	Above 70% for 5 minutes	Notify `internship-alerts`
Instance unhealthy	`StatusCheckFailed`	1 or more	Notify `internship-alerts`

Optionally, push the Apache access log to CloudWatch Logs with the CloudWatch agent. Keep volume small and set a short retention period (for example 7 days) to stay inside Free Tier ingestion limits.

Alarm test

1. Install a stress tool and run a short CPU load on the instance.
2. Watch the CPU alarm move to the In alarm state and confirm the email arrives.
3. Stop the load immediately and confirm the alarm returns to OK.

```
# Short CPU stress for the alarm demo, stop right after the test
sudo apt install -y stress
stress --cpu 2 --timeout 360
```

AWS Budget

1. Open Billing → Budgets and create a cost budget.
2. Set alerts at $1 and $3 estimated charges, or at Free Tier usage thresholds.
3. Send the notifications to my email and confirm they are received.
4. Check the Billing dashboard at least twice a week during the project.

Evidence to capture: alarm configuration, alarm history, the SNS confirmation and alert emails, and the Budget page.

9. Phase 6: Log Analysis and Validation

A 200–1000 row sample dataset and a short Python script answer four questions about web traffic without any paid analytics tool.

Dataset

The file `web-access-logs.csv` has these fields:

Field	Example
timestamp	2026-07-20 10:15:22
ip	203.0.113.45
method	GET
path	/index.html
status	200
bytes	2450
user_agent	Mozilla/5.0

First rows of the file:

```
timestamp,ip,method,path,status,bytes,user_agent
2026-07-20 10:15:22,203.0.113.45,GET,/index.html,200,2450,Mozilla/5.0
2026-07-20 10:16:05,198.51.100.7,GET,/health,200,120,curl/8.5.0
2026-07-20 10:17:41,203.0.113.45,GET,/missing.html,404,512,Mozilla/5.0
```

Analysis script

The script `analyze_logs.py` reports top pages, 4xx and 5xx counts, the peak traffic hour, and unique visitor IPs.

```
import csv
from collections import Counter

pages, hours, ips = Counter(), Counter(), set()
errors_4xx = errors_5xx = total = 0

with open("web-access-logs.csv", newline="") as f:
```

```
    for row in csv.DictReader(f):
        total += 1
        pages[row["path"]] += 1
        hours[row["timestamp"][11:13]] += 1
        ips.add(row["ip"])
        status = int(row["status"])
        if 400 <= status < 500:
            errors_4xx += 1
        elif status >= 500:
            errors_5xx += 1

peak_hour, peak_count = hours.most_common(1)[0]

print(f"Total requests: {total}")
print("Top requested pages:")
for path, count in pages.most_common(5):
    print(f"  {path}: {count}")
print(f"4xx errors: {errors_4xx}")
print(f"5xx errors: {errors_5xx}")
print(f"Peak traffic hour: {peak_hour}:00 ({peak_count} requests)")
print(f"Unique visitor IPs: {len(ips)}")
```

Run it with `python3 analyze_logs.py` and capture a screenshot of the output for the deliverables.

Validation tests

Test	Expected result
Website opened from browser and `curl`	Page loads with HTTP 200
SSH from a non-allowed IP	Connection times out
`aws s3 ls s3://my-internship-bucket` from EC2	Lists the project bucket
`aws s3 ls s3://some-other-bucket` from EC2	`AccessDenied` (IAM denial case)
CPU stress test	Alarm fires and email arrives
Log sync to S3	Files appear under `logs/yyyy-mm-dd/`

Cleanup

1. Stop or terminate the EC2 instance once evidence is captured.
2. Delete unused volumes, extra snapshots, and test S3 objects.
3. Delete the VPC components if the project is finished.
4. Filter by the `Project=InternshipAWS` tag to confirm nothing is left running.
5. Check the Billing dashboard one last time.

10. Free Tier Safety, Risks, and Deliverables

Cost control depends on a few habits: one small instance, no paid-by-the-hour services, tagged resources, and regular billing checks.

Free Tier safety checklist

- [ ] Only one micro EC2 instance is running
- [ ] No NAT Gateway, ALB, or RDS created
- [ ] S3 stores only small demo files and logs
- [ ] CloudWatch log ingestion kept minimal
- [ ] Snapshot count limited (one demo snapshot)
- [ ] Budget alert configured and email confirmed
- [ ] Resources tagged for easy cleanup
- [ ] Unused resources stopped or terminated after the demo
- [ ] Billing dashboard checked at least twice a week

Risks and mitigation

Risk	Impact	Mitigation
Unexpected AWS charges	High	Stick to Free Tier services, enable Budgets, avoid NAT/ALB/RDS
SSH open to the world	High	Restrict port 22 to my IP only
Over-permissioned IAM	Medium	Least privilege and bucket-scoped policies
Too many logs in CloudWatch	Medium	Limit retention and upload volume

Risk	Impact	Mitigation
Forgotten resources	Medium	Tag everything and follow the cleanup checklist

Deliverables checklist

- [ ] Project documentation (this document)
- [ ] Architecture diagram
- [ ] IAM evidence: users, groups, roles, policies (secrets redacted)
- [ ] VPC, subnet, and route table screenshots
- [ ] EC2 and web app proof (browser and `curl`)
- [ ] S3 bucket structure and object list
- [ ] `web-access-logs.csv` and analysis output
- [ ] CloudWatch alarm and SNS email screenshots
- [ ] EBS snapshot proof and restore notes
- [ ] Budget alert screenshot
- [ ] Free Tier safety checklist filled
- [ ] Project explanation video (5–10 minutes)

Conclusion

The project delivers a working AWS environment in which networking, IAM, compute, storage, and monitoring are connected and tested. It shows least-privilege access through an instance role, a working alert and backup flow, and practical log analysis on a small dataset, all designed to stay within the Free Tier.